INTERNSHIP PROJECT REPORT

Disaster Tweet Classification Using NLP and Machine Learning

Internship Organization: Nexthikes

Submitted by: Raghav Arora

Abstract

Social media platforms can spread information rapidly during emergencies. However, a tweet that contains words such as “disaster,” “fire,” or “flood” does not always describe a real-world emergency. This project develops a binary text-classification system that predicts whether a tweet is disaster-related or non-disaster. The implementation uses TF-IDF to convert text into numerical features and Logistic Regression for classification. A Flask web application allows a user to enter a tweet and view the predicted category. On the recorded held-out test set, the model achieved 81.42% accuracy. The result indicates that the approach provides a useful baseline, while the class-wise recall shows that further improvement is possible, especially for disaster-related tweets.

1. Introduction

During floods, earthquakes, wildfires, and other emergencies, social media may contain timely reports from people in affected areas. Manually reviewing large numbers of posts is difficult, and automated text classification can help organize this information for further review. The main challenge is contextual meaning: users may use disaster-related vocabulary figuratively, sarcastically, or in unrelated situations. For example, “The exam was a disaster” does not necessarily indicate a real emergency.

This project applies Natural Language Processing (NLP) and supervised machine learning to classify tweet text into two categories. It also provides a small web interface so that a user can submit a new tweet without directly running model code.

2. Problem Statement

The objective is to build a model that can distinguish tweets describing real disaster events from tweets that are not related to actual disasters. The system should process text, predict one of two classes, and present the prediction through a simple web

interface. Since the task depends on language and context, the model may still make errors on ambiguous or metaphorical tweets.

3. Project Objectives

- Inspect and prepare a labelled tweet dataset for supervised learning.
- Represent tweet text numerically using TF-IDF features.
- Train a Logistic Regression classifier for binary classification.
- Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Save the trained model and fitted vectorizer so they can be reused.
- Build a Flask web application that accepts tweet text and displays the predicted class.

4. Dataset Description

The dataset used during development contains 7,613 records and five columns: id, keyword, location, text, and target. The `text` column contains the tweet and the `target` column contains its class label. In this project, target 0 represents a non-disaster tweet and target 1 represents a disaster-related tweet. The checked dataset had no missing values in the text or target columns. Missing values were present in the keyword and location columns, but those columns were not required for the current model.

Column	Purpose	Used by model?
id	Tweet identifier	No
keyword	Associated keyword	No
location	Location information	No
text	Tweet content	Yes — input
target	Class label (0 or 1)	Yes — output

5. Tools and Technologies

- Python: core programming language.
- Pandas: loading and inspecting the CSV dataset.
- Scikit-learn: train-test split, TF-IDF vectorization, Logistic Regression, and evaluation metrics.

- Joblib: saving and loading the trained model and vectorizer.
- Matplotlib: generating the confusion matrix image.
- Flask: connecting the model to a web application.
- HTML and CSS: building and styling the user interface.

6. Methodology

6.1 Dataset inspection

The CSV file was loaded with Pandas and inspected for its dimensions, column names, sample rows, and missing values. The model uses only the tweet text and target label, so the remaining metadata columns are not included in the current training pipeline.

6.2 Train-test split

The labelled examples were split into training and testing sets using an 80:20 ratio. Stratification was used to preserve the approximate class distribution in both subsets, and a fixed random state was set to make the split reproducible.

6.3 TF-IDF feature extraction

Machine learning models require numerical inputs. Term Frequency–Inverse Document Frequency (TF-IDF) converts the tweet text into weighted numerical features. The current implementation uses English stop-word filtering, a maximum of 10,000 features, and unigrams and bigrams. The vectorizer is fitted on the training text only, then applied to the test text.

6.4 Logistic Regression model

Logistic Regression was selected as a simple baseline classifier for the binary task. It learns a relationship between the TF-IDF features and the target labels, then predicts whether an unseen tweet belongs to class 0 or class 1.

6.5 Evaluation

The model was evaluated on the held-out test set using accuracy, precision, recall, and F1-score. A confusion matrix was also generated to show correct and incorrect predictions for each class. These measures provide more information than accuracy alone, because false negatives and false positives have different implications for disaster detection.

6.6 Model saving and web integration

After training, the classifier was saved as `disaster_model.pkl` and the fitted TF-IDF vectorizer as `tfidf_vectorizer.pkl`. The Flask backend loads both files, transforms the

user's input tweet using the saved vectorizer, obtains the prediction from the model, and passes the result to the HTML template for display.

7. System Workflow

Dataset (tweets.csv) → Select text and target → Split into training and testing data → TF-IDF feature extraction → Logistic Regression training → Model evaluation → Save model and vectorizer → Flask web form → Tweet prediction shown to the user.

8. Results and Evaluation

The recorded test evaluation used 1,523 tweets and achieved an overall accuracy of 81.42%. The class-wise results from the current run are shown below.

Class	Precision	Recall	F1-score
Non-disaster	0.80	0.90	0.85
Disaster	0.84	0.70	0.76
Macro average	0.82	0.80	0.81
Weighted average	0.82	0.81	0.81

The model identifies non-disaster tweets more consistently than disaster tweets in this test run. Disaster precision is 0.84, while disaster recall is 0.70. This means some actual disaster tweets are missed, so the model should be treated as a prototype and not as a stand-alone emergency alert system.

9. Confusion Matrix

Insert the generated `confusion_matrix.png` image in the space below. In Microsoft Word, use Insert → Pictures → This Device, select `confusion_matrix.png`, and resize it to fit within the page margins.

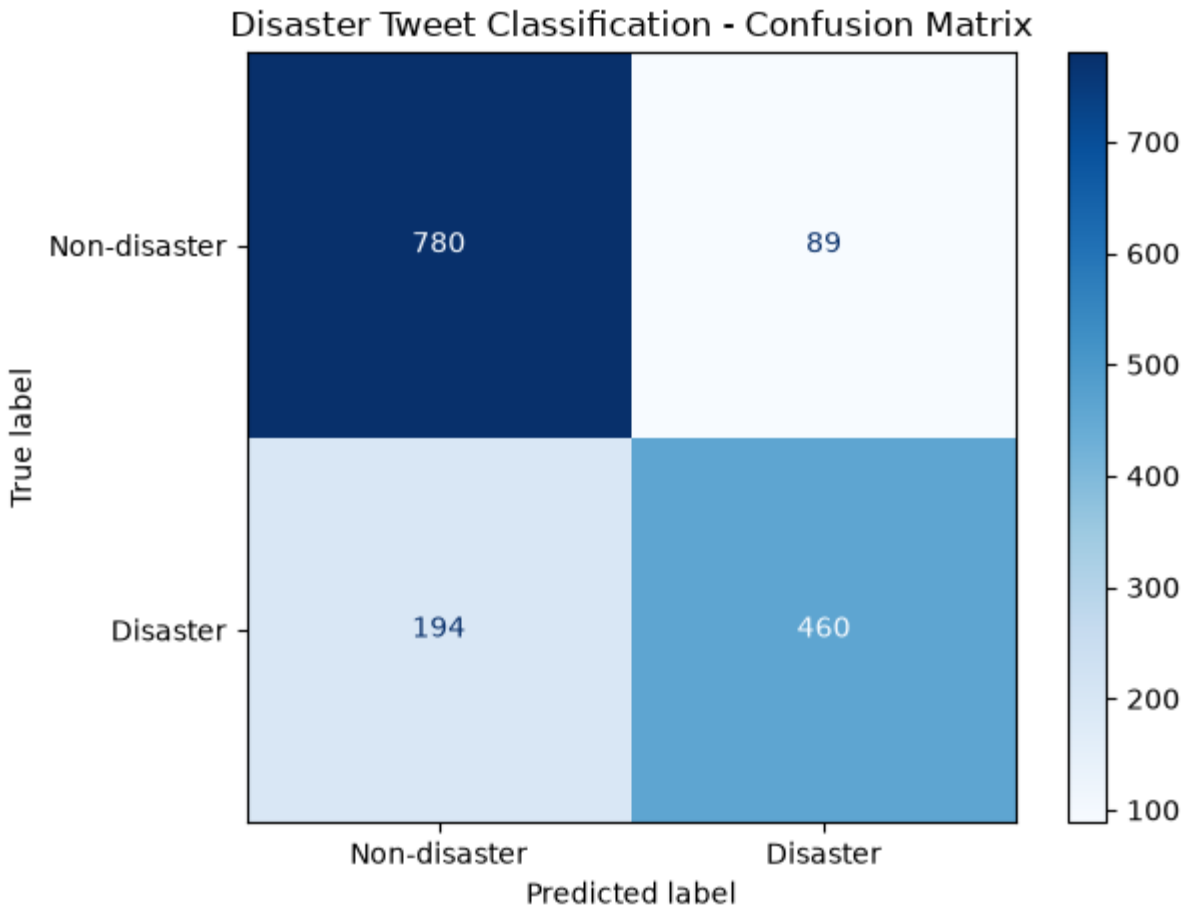


Figure 1. Confusion matrix for the test dataset.

10. Web Application

The web interface is built with Flask and HTML/CSS. A user enters a tweet into the text area and submits the form. The backend loads the saved model and vectorizer, converts the tweet into TF-IDF features, predicts the class, and displays either “Disaster-related” or “Non-disaster.” The interface provides a simple way to test the classifier without using the Python training script.

The current application performs prediction on user-entered text only. It does not connect to Twitter/X or continuously monitor social media, and its output should not be interpreted as verified emergency information.

11. Limitations

- The model depends on the quality and coverage of the labelled dataset.
- Sarcasm, metaphor, slang, and context-dependent expressions may be misclassified.
- The current baseline uses TF-IDF and Logistic Regression rather than contextual language models.
- The application does not perform real-time social media monitoring.
- Test performance does not guarantee the same performance on tweets from different time periods or sources.

12. Future Scope

- Compare Logistic Regression with Multinomial Naive Bayes and other suitable classifiers.
- Tune model parameters and analyze false positives and false negatives.
- Explore contextual embeddings or transformer-based models for improved language understanding.
- Add prediction confidence information with appropriate caveats.
- Deploy the Flask application to a hosting platform if required.

13. Conclusion

The Disaster Tweet Classification project demonstrates a complete introductory NLP workflow, from loading and preparing labelled data to training, evaluating, saving, and using a machine learning model in a web application. The current Logistic Regression model achieved 81.42% accuracy on the held-out test set. Although the results provide a useful baseline, the lower recall for disaster tweets highlights an area for future improvement. The project strengthened practical understanding of text vectorization, supervised classification, evaluation metrics, and Flask-based model integration.

14. Project Files

- `tweets.csv` — labelled tweet dataset.
- `check_dataset.py` — dataset inspection script.
- `train_model.py` — model training and evaluation script.
- `disaster_model.pkl` — saved Logistic Regression model.
- `tfidf_vectorizer.pkl` — saved TF-IDF vectorizer.
- `confusion_matrix.png` — evaluation visualization.
- `app.py` — Flask backend.
- `templates/index.html` — web interface.